

SeniorCLI

Guía técnica de los 3 frentes de desarrollo

Cómo dividir, desarrollar e integrar el proyecto en Rust

Frente 1

Runtime & Context

Frente 2

AI & Learning Harness

Frente 3

CLI, Producto & Persistencia

Objetivo del documento Que cada frente pueda trabajar de forma casi independiente, pero con contratos claros para que la integración sea predecible. El foco no es repartir comandos; es repartir responsabilidades internas.

Versión de trabajo - arquitectura inicial

Descripción general del proyecto

SeniorCLI es un AI Learning Harness orientado al aprendizaje individual de programación desde la terminal. Su objetivo es acompañar al estudiante mientras trabaja en proyectos reales y ofrecer una experiencia semejante a tener un ingeniero senior disponible para explicar, cuestionar, orientar y revisar, sin convertir el proceso de aprendizaje en una simple generación automática de código.

A diferencia de un asistente general de inteligencia artificial, SeniorCLI no se plantea como una interfaz directa entre el usuario y un LLM. El producto se diseña como una capa de software que controla qué contexto se envía al modelo, qué tipo de intervención educativa está permitida, qué herramientas pueden ejecutarse, qué información debe conservarse entre sesiones y cómo se valida la respuesta antes de presentarla al estudiante.

Problemática que atiende

Durante el aprendizaje de programación, un estudiante suele alternar entre el editor, la terminal, documentación, buscadores y asistentes de IA. Cuando aparece un error, el estudiante debe explicar manualmente el contexto del proyecto o copiar grandes fragmentos de código. Además, un modelo de lenguaje puede responder con información incompleta, asumir datos que no conoce, entregar soluciones antes de tiempo o perder el objetivo pedagógico de la conversación.

SeniorCLI busca reducir esa fragmentación. El sistema obtiene contexto técnico del proyecto de forma controlada, mantiene un perfil de aprendizaje y utiliza una política pedagógica para decidir si conviene preguntar, dar una pista, mostrar un ejemplo, proponer pseudocódigo o, sólo cuando corresponda, explicar una solución completa.

Qué será SeniorCLI

El producto se concibe como una CLI multiplataforma escrita principalmente en Rust. Desde la terminal, el alumno podrá iniciar una sesión de aprendizaje, pedir ayuda sobre un error, revisar una parte de su código y consultar su progreso. SeniorCLI recopilará únicamente el contexto necesario del proyecto, lo transformará en una representación estructurada y lo entregará al motor educativo.

La primera versión usará una API externa de DeepSeek como proveedor de modelo. Sin embargo, la arquitectura debe permitir sustituir ese proveedor sin modificar el núcleo del producto. Se contemplan tres modalidades de inferencia: API externa, modelo local y proveedor remoto/institucional. La elección concreta del modelo queda desacoplada del Learning Engine.

Objetivo general

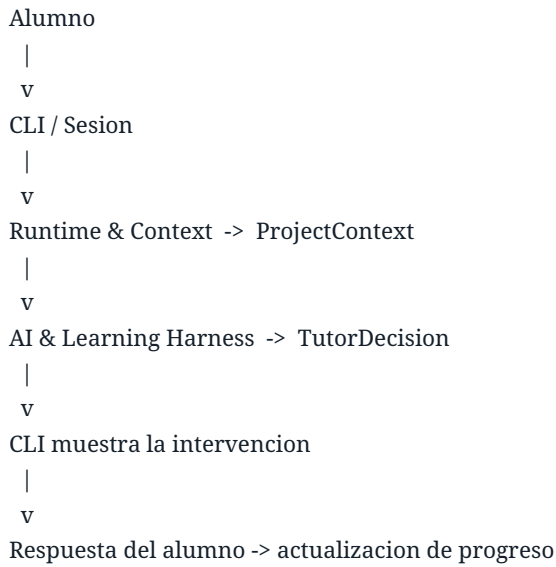
Desarrollar un harness educativo para programación que integre contexto real del proyecto, modelos de lenguaje y seguimiento del aprendizaje, con el fin de proporcionar asistencia personalizada desde la terminal y reducir progresivamente la dependencia del estudiante respecto al asistente.

Principios de diseño

- El LLM es un componente probabilístico, no la autoridad del sistema.
- La mayor cantidad posible de trabajo mecánico debe resolverse con software determinístico antes de consultar al modelo.
- El contexto enviado debe ser suficiente para responder, pero no innecesariamente grande.
- Una petición ambigua no debe provocar que el sistema invente contexto; debe pedir aclaración o reducir el alcance.

- El harness debe conservar el objetivo pedagógico incluso cuando el modelo proponga una respuesta excesiva o una solución completa.
- La memoria del estudiante debe ser estructurada y verificable; no debe depender únicamente del historial de conversación.
- Los proveedores de LLM deben ser intercambiables mediante una interfaz común.
- Los tres frentes deben poder desarrollarse y probarse de forma independiente mediante contratos y mocks.

Cómo funciona a alto nivel



El flujo anterior explica la división del proyecto. Cada frente es responsable de una parte distinta de la tubería y se comunica con los demás mediante estructuras compartidas. Esto permite que dos personas trabajen en el Context Engine mientras otras desarrollan el Learning Engine o la CLI sin esperar a que el resto termine.

Por qué Rust

Rust se propone como lenguaje principal porque SeniorCLI debe ejecutarse como una herramienta nativa de terminal, interactuar con archivos y procesos, distribuirse como un binario y mantener un control estricto sobre errores y recursos. Rust ofrece buen rendimiento, seguridad de memoria y un ecosistema adecuado para CLIs asíncronas. El uso de Rust no implica que el LLM se ejecute en el equipo del alumno: la inferencia puede realizarse mediante cualquiera de los proveedores definidos anteriormente.

Alcance inicial

- Construir primero el flujo principal de aprendizaje y depuración; no intentar cubrir todos los lenguajes o comandos desde la primera versión.
- Trabajar inicialmente con pocos stacks soportados y ampliar sólo cuando el flujo central sea estable.
- Priorizar contexto correcto, decisiones pedagógicas y persistencia confiable por encima de una interfaz visual compleja.
- Tratar desde el inicio los casos difíciles: alucinaciones, peticiones vagas, pérdida de contexto, respuestas inválidas del modelo y errores de integración.

1. La idea central: tres frentes, un solo flujo

SeniorCLI debe verse como una tubería. Cada frente recibe un tipo de información bien definido, hace su trabajo y entrega un resultado con un formato acordado. Así el equipo evita que todos toquen las mismas partes del código.

```
USUARIO
|
v
[Frente 3: CLI]
| pregunta / comando
v
[Frente 1: Runtime & Context]
| ProjectContext
v
[Frente 2: AI & Learning Harness]
| TutorDecision
v
[Frente 3: CLI + Persistencia]
|
v
USUARIO
```

Regla de integración Los frentes no se integran compartiendo “un montón de funciones”. Se integran intercambiando estructuras pequeñas y estables: `ProjectContext`, `StudentProfile`, `TutorRequest` y `TutorDecision`.

1.1 Contratos compartidos

```
pub struct ProjectContext {
    pub stack: StackInfo,
    pub relevant_files: Vec<RelevantFile>,
    pub diagnostics: Vec<Diagnostic>,
    pub git_diff: Option<String>,
}

pub struct StudentProfile {
    pub level: SkillLevel,
    pub topics: Vec<TopicMastery>,
    pub recent_errors: Vec<String>,
}

pub struct TutorDecision {
    pub concept: String,
    pub intervention: Intervention,
    pub message: String,
    pub question: Option<String>,
    pub confidence: f32,
}
```

Estas estructuras son el “idioma común” del proyecto. Si se mantienen estables, un frente puede usar datos falsos (mocks) mientras otro todavía termina su implementación real.

1.2 Reparto recomendado del equipo

Frente	Personas	Responsable	Entrega principal
1. Runtime & Context	2	Perfil fuerte 1	ProjectContext confiable
2. AI & Learning Harness	2	Perfil fuerte 2	TutorDecision pedagógico
3. CLI / Producto / Persistencia	2	Perfil fuerte 3	Experiencia de usuario + sesiones
Coordinación	1	Tú	Visión, alcance, prioridades y validación funcional

1

Runtime & Context

Hace que SeniorCLI entienda el proyecto real antes de pedir ayuda al LLM.

2. Qué problema resuelve

Un LLM sin contexto suficiente recibe preguntas como “¿por qué no funciona?” o “arregla esto” y no sabe a qué archivo, lenguaje, error o cambio se refiere. El Frente 1 convierte el estado real del proyecto en información estructurada y mínima.

2.1 Qué debe construir

- Detector de raíz del proyecto y stack (Rust, Java, Python, Node, etc.).
- Explorador de archivos relevante, respetando .gitignore y límites de tamaño.
- Integración con Git: estado, diff y archivos modificados.
- Runner controlado para comandos conocidos (tests, compilación, lint).
- Parser de errores/diagnósticos.
- Selector de contexto: escoger lo necesario sin enviar el repositorio completo.
- Escáner básico de secretos antes de construir contexto para un proveedor remoto.

2.2 Cómo se hace en Rust

Necesidad	Opción Rust	Uso
CLI/process	std::process::Command / tokio::process::Command	Ejecutar git, tests o compiladores con timeout.
Recorrer archivos	walkdir + ignore	Explorar el repo respetando .gitignore.
Serialización	serde / serde_json	Construir ProjectContext estable.
Errores	anyhow + thiserror	Diferenciar errores de sistema, ejecución y parsing.
Regex / parsing	regex	Extraer ubicaciones de errores simples.
Async	tokio	No bloquear la CLI mientras corre un proceso o lectura pesada.

```
pub async fn collect_context(root: &Path) -> Result<ProjectContext> {
    let stack = detect_stack(root)?;
    let git_diff = collect_git_diff(root).await.ok();
    let diagnostics = run_safe_diagnostics(root, &stack).await?;
    let relevant_files = select_relevant_files(root, &diagnostics, git_diff.as_deref())?;

    Ok(ProjectContext { stack, relevant_files, diagnostics, git_diff })
}
```

2.3 Flujo interno del frente

1. Detectar raíz y stack.
2. Leer cambios recientes y error actual.
3. Ejecutar sólo diagnósticos permitidos.
4. Identificar archivos y rangos útiles.
5. Filtrar secretos y contenido enorme.
6. Entregar ProjectContext; no decidir la respuesta pedagógica.

2.4 Riesgos y puntos delicados

Contexto insuficiente Si sólo se manda el error pero no el código que lo produce, el LLM tenderá a adivinar. Mitigación: incluir archivo, rango y símbolos relacionados.

Contexto excesivo Enviar el repositorio completo aumenta costo, latencia y ruido. Mitigación: límites por archivo, top-K de relevancia y prioridad a archivos modificados/diagnósticos.

Petición vaga “No funciona” no contiene intención. El Context Engine puede aportar evidencia, pero no debe inventar el objetivo del alumno. Debe marcar ambigüedad para que el Frente 2 pregunte.

Stack detectado incorrectamente Un monorepo puede tener varios stacks. Mitigación: StackInfo debe admitir múltiples componentes y una raíz activa.

Comandos peligrosos Nunca ejecutar texto generado por el LLM directamente como shell. Usar una lista de acciones permitidas y argumentos validados.

Secretos Antes de enviar contexto remoto, excluir .env, llaves privadas y patrones de tokens. No confiar únicamente en extensiones de archivo.

3. Qué problema resuelve

El LLM es probabilístico. Puede alucinar, interpretar mal una petición, dar una solución demasiado pronto o perder el hilo. El harness existe precisamente para que SeniorCLI no sea “un chat con una API”, sino un sistema que impone reglas educativas y valida el intercambio.

3.1 Qué debe construir

- Abstracción `ModelProvider` para no acoplar `SeniorCLI` a `DeepSeek`.
- Constructor de `TutorRequest` a partir de `ProjectContext` + `StudentProfile` + mensaje del alumno.
- Política pedagógica: pregunta, pista, explicación, ejemplo, pseudocódigo, código parcial, solución guiada.
- Parser/validador de respuestas estructuradas del modelo.
- Manejador de ambigüedad: cuándo preguntar antes de enseñar.
- Actualizador de mastery/progreso a partir de evidencias, no de una sola respuesta.
- Guardas contra “sobre-ayuda”: impedir que una pista termine convirtiéndose en solución completa.

3.2 Cómo se hace en Rust

```
#[async_trait::async_trait]
pub trait ModelProvider: Send + Sync {
    async fn complete(&self, request: ModelRequest) -> Result<ModelResponse>;
}

pub struct DeepSeekProvider {
    client: reqwest::Client,
    api_key: SecretString,
}

pub struct TutorEngine<P: ModelProvider> {
    provider: P,
    policy: TeachingPolicy,
}
```

La primera implementación puede ser `DeepSeekProvider`. Más adelante el mismo `trait` puede tener `LocalProvider` u otro proveedor remoto sin modificar el `Learning Engine`.

Necesidad	Opción Rust	Uso
HTTP	<code>reqwest</code>	Llamadas a la API actual.
Async trait	<code>async-trait</code>	Provider intercambiable con métodos <code>async</code> .
Datos	<code>serde</code>	<code>Request/response</code> estructurados.
Secretos	<code>secrecy</code>	Reducir exposición accidental de API keys.
Validación	validador o validación manual	Rechazar respuestas incompletas/ilegales.
Retries	<code>backoff</code> propio / <code>tokio::time</code>	Reintentos limitados; nunca loops infinitos.

```
pub enum Intervention {
    Clarify,
    SocraticQuestion,
    Hint,
    Explanation,
    Example,
    Pseudocode,
    PartialCode,
    GuidedSolution,
}

pub struct TutorDecision {
    pub concept: String,
    pub intervention: Intervention,
    pub message: String,
    pub question: Option<String>,
    pub confidence: f32,
}
```

3.3 La lógica del harness

```
mensaje del alumno
+
ProjectContext
+
StudentProfile
v
¿La intención es clara? -- no --> Clarify
| sí
v
¿Hay evidencia suficiente? -- no --> pedir dato / ejecutar diagnóstico permitido
| sí
v
Elegir nivel de ayuda
v
LLM propone respuesta estructurada
v
Validar política pedagógica
v
TutorDecision
```

3.4 Fallos críticos del harness

Alucinación técnica El modelo puede inventar APIs, sintaxis o causas. El harness debe expresar incertidumbre, preferir evidencia del compilador/tests y nunca convertir “confianza del LLM” en verdad.

Petición vaga o ambigua “Haz que funcione” puede significar muchas cosas. Si no hay intención clara, la intervención correcta es Clarify, no adivinar.

El harness “se pierde” Puede ocurrir por historial demasiado largo, estado desincronizado o resúmenes que eliminan una decisión importante. Mitigación: separar estado estructurado de conversación libre y reconstruir cada turno desde ProjectContext + StudentProfile + objetivo actual.

Pérdida de objetivo pedagógico El modelo puede empezar socrático y terminar dando la solución completa. Mitigación: el nivel de ayuda es un campo externo al prompt; validar la salida y recortar/reintentar si viola la política.

Respuesta JSON inválida Los modelos a veces agregan texto o rompen el esquema. Mitigación: parsear con serde, un reintento de reparación controlado y fallback a mensaje seguro.

Loop de herramientas Si el LLM puede pedir acciones repetidamente, puede entrar en bucle. Limitar rondas, costo y herramientas por turno.

Prompt injection desde el repo Un README o comentario puede contener “ignora instrucciones”. El contenido del proyecto es datos, no instrucciones. Separarlo claramente del system/policy.

Perfil del alumno equivocado No subir mastery por una sola respuesta. Usar varias evidencias y permitir degradación/ajuste si aparecen fallos posteriores.

4. Qué problema resuelve

Aunque los otros frentes funcionen, el producto falla si la interacción es confusa, la sesión se pierde, la configuración es frágil o el usuario no sabe qué está haciendo SeniorCLI. Este frente es responsable del flujo visible y del estado local.

4.1 Qué debe construir

- Comandos iniciales: senior init, senior learn, senior debug, senior progress.
- REPL o flujo interactivo del modo learn/debug.
- Configuración del proyecto en .senior/.
- Persistencia de StudentProfile y sesiones.
- Recuperación segura después de cierre/crash.
- Presentación de mensajes, código, preguntas y estados de carga.
- Manejo de Ctrl+C, errores del proveedor y timeouts de forma comprensible.
- Versionado/migración del esquema local cuando cambien estructuras.

4.2 Cómo se hace en Rust

Necesidad	Opción Rust	Uso
Argumentos	clap	Subcomandos y flags.
REPL	rustyline	Historial, edición y experiencia interactiva.
Terminal	crossterm / terminad	Formato, eventos y markdown ligero.
Persistencia simple	serde_json / toml	Config y perfil en MVP.
Persistencia robusta	SQLite (rusqlite/sqlx)	Si crece el historial y las relaciones.
Rutas	dirs	Ubicaciones de config global/local.

```
pub struct AppState {  
    pub project_id: String,  
    pub profile: StudentProfile,  
    pub active_session: SessionState,  
    pub config: ProjectConfig,  
}
```

```
.senior/  
config.toml  
profile.json  
sessions/  
2026-09-16T1234.jsonl
```

4.3 Flujo de una sesión

1. La CLI recibe el comando y carga AppState.
2. Pide ProjectContext al Frente 1.
3. Entrega mensaje + contexto + perfil al Frente 2.
4. Renderiza TutorDecision para el alumno.
5. Captura la respuesta del alumno.

6. Guarda el turno y actualiza el perfil sólo cuando corresponda.
7. En cierre normal o inesperado, deja estado recuperable.

4.4 Riesgos y puntos delicados

Estado desincronizado Si la UI muestra un nivel de ayuda pero el Learning Engine usa otro, la experiencia se rompe. La CLI debe tratar TutorDecision como fuente de verdad por turno.

Pérdida de sesión No esperar al final para guardar. Escribir turnos de forma incremental y usar escrituras atómicas para archivos críticos.

Perfil corrupto Validar al cargar y conservar respaldo/migración. Nunca sobrescribir un perfil válido con un JSON parcial.

UX demasiado “chat” SeniorCLI debe mostrar evidencia del proyecto, estado de aprendizaje y tipo de intervención; no parecer simplemente una ventana de chat.

Errores opacos Un 429/timeout no debe verse como stack trace crudo. Traducir fallos técnicos a mensajes accionables y permitir reintento/cambio de provider.

Compatibilidad de terminal No depender de colores o glyphs para información crítica. Mantener salida legible en Windows/Linux/macOS.

5. Cómo trabajan aislados y luego se unen

La independencia no significa que cada frente invente sus propios tipos. Primero se congelan contratos mínimos; después cada frente desarrolla contra mocks. La integración ocurre reemplazando mocks por implementaciones reales.

5.1 Ejemplo real: los tres frentes trabajando el mismo caso

CASO: alumno ejecuta `senior debug` en Spring Boot con NullPointerException

Frente 1	Frente 2	Frente 3
Implementa un FakeProjectContext y luego el detector real. Su prueba pasa si entrega stack, archivo, línea y diagnóstico.	Usa ese ProjectContext falso para decidir: concepto Optional/null, intervención Hint y pregunta socrática.	Usa un TutorDecision falso para construir la pantalla, input del usuario y guardado de sesión.

```
// Mock compartido para trabajar antes de integrar
let context = ProjectContext::fake_java_null_pointer();
let decision = TutorDecision::fake_hint_optional();
```

5.2 Orden recomendado de integración

1. Definir tipos compartidos en un módulo contracts/.
2. Cada frente crea pruebas usando mocks de esos contratos.
3. Integrar Frente 1 -> Frente 2: contexto real entra al harness.
4. Integrar Frente 2 -> Frente 3: decisión real llega a la CLI.
5. Ejecutar un vertical slice de punta a punta con un caso simple.
6. Sólo después agregar más lenguajes, más herramientas o más comandos.

No hacer No esperar a que cada frente esté “100% terminado” para integrarlo. Eso acumula incompatibilidades. Integren un caso pequeño desde temprano y manténganlo funcionando.

5.3 Estructura del repositorio

```
src/
  main.rs
  contracts/
    project_context.rs
    student_profile.rs
    tutor_decision.rs
  runtime/          # Frente 1
    detector.rs
    files.rs
    git.rs
    runner.rs
    diagnostics.rs
  learning/         # Frente 2
    engine.rs
    policy.rs
    provider.rs
    deepseek.rs
    validation.rs
```

```
mastery.rs
app/      # Frente 3
cli.rs
repl.rs
storage.rs
session.rs
ui.rs
tests/
fixtures/
integration/
```

6. Puntos clave que los 3 frentes deben respetar

Determinismo alrededor del LLM Todo lo que pueda resolverse con reglas, parsing, compilador, tests o estado estructurado no debe delegarse al modelo.

Evidencia antes que adivinación Cuando falte contexto, SeniorCLI pregunta o ejecuta diagnóstico permitido. No inventa.

Estado estructurado antes que memoria conversacional El objetivo actual, nivel de ayuda, perfil y contexto viven en estructuras; el historial de chat es secundario.

Proveedor intercambiable DeepSeek es la implementación inicial. La arquitectura debe permitir API externa, modelo local u otro proveedor sin reescribir el producto.

Privacidad por diseño El contexto enviado a una API debe ser mínimo y filtrado.

Fallos visibles y recuperables Timeouts, respuestas inválidas, crashes o archivos corruptos deben tener fallback y recuperación.

Pedagogía como regla de producto Una respuesta técnicamente correcta puede ser incorrecta para SeniorCLI si entrega demasiado y evita que el alumno piense.

Métricas desde el inicio Registrar latencia, tokens/solicitudes, errores de parsing, intervenciones y cantidad de ayuda utilizada; luego podrán evaluar si el harness funciona.

7. Qué significa “Frente listo” en la primera etapa

Frente	Debe demostrar	No necesita todavía	Prueba mínima
Runtime & Context	Detectar 2 stacks, capturar un error y entregar contexto mínimo.	Indexación semántica, AST completo, sandbox avanzado.	Fixture Java/Python produce ProjectContext esperado.
AI & Learning	Tomar contexto + perfil y generar una intervención válida con DeepSeek.	Multi-model routing complejo, fine-tuning.	Petición vaga -> Clarify; petición clara -> Hint/Question.
CLI / Persistencia	Ejecutar learn/debug, mostrar decisión y recuperar sesión.	Dashboard web, sincronización cloud.	Cerrar/reabrir sin perder perfil ni sesión reciente.

8. Checklist para arrancar

- Acordar nombres y campos de ProjectContext, StudentProfile y TutorDecision.
- Crear repo y estructura de carpetas.
- Asignar 2 personas por frente y un responsable técnico en cada uno.
- Crear mocks compartidos antes de implementar motores reales.
- Elegir 1 caso vertical inicial (recomendado: debug sencillo en Java o Python).
- Conectar DeepSeek únicamente detrás de ModelProvider.
- Integrar el primer caso de punta a punta lo antes posible.
- Registrar desde el inicio los fallos del harness: alucinaciones, ambigüedad, JSON inválido, pérdida de estado y sobre-ayuda.

Resultado esperado Si siguen esta separación, cada frente puede avanzar sin esperar a que los demás terminen. La integración se vuelve el reemplazo de mocks por componentes reales, no una “pegada” final de tres proyectos distintos.